Assignment-12.3(AI)

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

- **Correctly sorted student records.**
- **Performance comparison between Quick Sort and Merge Sort.**
- **Clear output of top-performing students.**

File Edit Selection View Go Run Terminal Help

AI

EXPLORER

- AI
  - age and email using conditional statements.py
  - ai (6.3 task - 5).py
  - ai (10.3 task -2).py
  - ai (10.4 task).py
  - ai (11.1).py
  - ai 9.1 (task-5).py
  - ai 9.1(task-1).py
  - ai 9.1(task-2).py
  - ai 9.1(task-3).py
  - ai 9.1(task-4.py
  - ai 9.3(task-1).py
  - ai 9.3(task-2).py
  - ai 9.3(task-3).py
  - ai 10.3 (task - 1).py
  - ai 10.3 (task - 3).py
  - ai ass.py
  - ai_lab(2.1).py
  - ai_lab(6.1).py
  - ai(6.3 task -2).py
  - ai(6.3 task-3).py
  - ai(6.3 task-4).py
  - ai(6.3).py
  - ai(7.1 task - 4).py
  - ai(7.1 task -2).py
  - ai(7.1 task-3).py
  - ai(7.1 task-5).py
  - ai(7.1) task-1.py
  - ai2.py
  - Allab(2.1).py
  - class Student with attributes.py
  - disaster_relief_data.txt
  - exam2.py
  - list are even and odd.py
  - simple bank account system.py
  - victim_records.txt

ai 9.1(task-3).py | ai 9.1(task-4.py | ai 9.1 (task-5).py | ai 10.3 (task - 1).py | ai (10.3 task -2).py | ai 10.

ai (11.1).py > ...

```python
# SR University Placement Cell - Student Record Sorting System
# This program stores student records and sorts them based on CGPA
# using Quick Sort and Merge Sort. It also compares their runtime.

import random
import time

# Function to create student records
def generate_students(n):
    print("\nGenerating student records...")
    students = []

    for i in range(n):
        name = "Student_" + str(i+1)
        roll = "SRU" + str(1000 + i)
        cgpa = round(random.uniform(5.0, 10.0), 2)

        students.append({"name": name, "roll": roll, "cgpa": cgpa})

    print("Student records generated successfully.\n")
    return students


# Quick Sort Algorithm
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2]["cgpa"]

    left = [x for x in arr if x["cgpa"] > pivot]
    middle = [x for x in arr if x["cgpa"] == pivot]
    right = [x for x in arr if x["cgpa"] < pivot]

    return quick_sort(left) + middle + quick_sort(right)


# Merge Sort Algorithm
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

File Edit Selection View Go Run Terminal Help

AI

EXPLORER

AI

- age and email using conditional statements.py
- ai (6.3 task - 5).py
- ai (10.3 task -2).py
- ai (10.4 task).py
- ai (11.1).py
- ai 9.1 (task-5).py
- ai 9.1(task-1).py
- ai 9.1(task-2).py
- ai 9.1(task-3).py
- ai 9.1(task-4.py
- ai 9.3(task-1).py
- ai 9.3(task-2).py
- ai 9.3(task-3).py
- ai 10.3 (task - 1).py
- ai 10.3 (task - 3).py
- ai ass.py
- ai_lab(2.1).py
- ai_lab(6.1).py
- ai(6.3 task -2).py
- ai(6.3 task-3).py
- ai(6.3 task-4).py
- ai(6.3).py
- ai(7.1 task - 4).py
- ai(7.1 task -2).py
- ai(7.1 task-3).py
- ai(7.1 task-5).py
- ai(7.1) task-1.py
- ai2.py
- Allab(2.1).py
- class Student with attributes.py
- disaster_relief_data.txt
- exam2.py
- list are even and odd.py
- simple bank account system.py
- victim_records.txt

OUTLINE

ai 9.1(task-3).py | ai 9.1(task-4.py | ai 9.1 (task-5).py | ai 10.3 (task - 1).py | ai (10.3 task -2).py | ai

ai (11.1).py > ...

```python
def merge_sort(arr):
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i]["cgpa"] > right[j]["cgpa"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# Display Top 10 Students
def display_top_students(students):
    print("\nTop 10 Students Based on CGPA")

    for i, student in enumerate(students[:10], start=1):
        print(i, "-", student["name"], "| Roll:", student["roll"], "| CGPA:", student["cgpa"])


# Main Program
print("SR University Placement Drive - Student Sorting System")

n = int(input("Enter number of student records to generate for testing: "))

students = generate_students(n)

# Copy lists so both algorithms get same data
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```python

n = int(input("Enter number of student records to generate for testing: "))

students = generate_students(n)

# Copy lists so both algorithms get same data
students_quick = students.copy()
students_merge = students.copy()


# Quick Sort Performance
print("Running Quick Sort...")
start = time.time()
quick_sorted = quick_sort(students_quick)
quick_time = time.time() - start
print("Quick Sort completed in", round(quick_time, 6), "seconds")


# Merge Sort Performance
print("\nRunning Merge Sort...")
start = time.time()
merge_sorted = merge_sort(students_merge)
merge_time = time.time() - start
print("Merge Sort completed in", round(merge_time, 6), "seconds")


# Performance Comparison
print("\nPerformance Comparison")
print("Quick Sort Time :", round(quick_time, 6), "seconds")
print("Merge Sort Time :", round(merge_time, 6), "seconds")


# Display Top Students
display_top_students(quick_sorted)
```

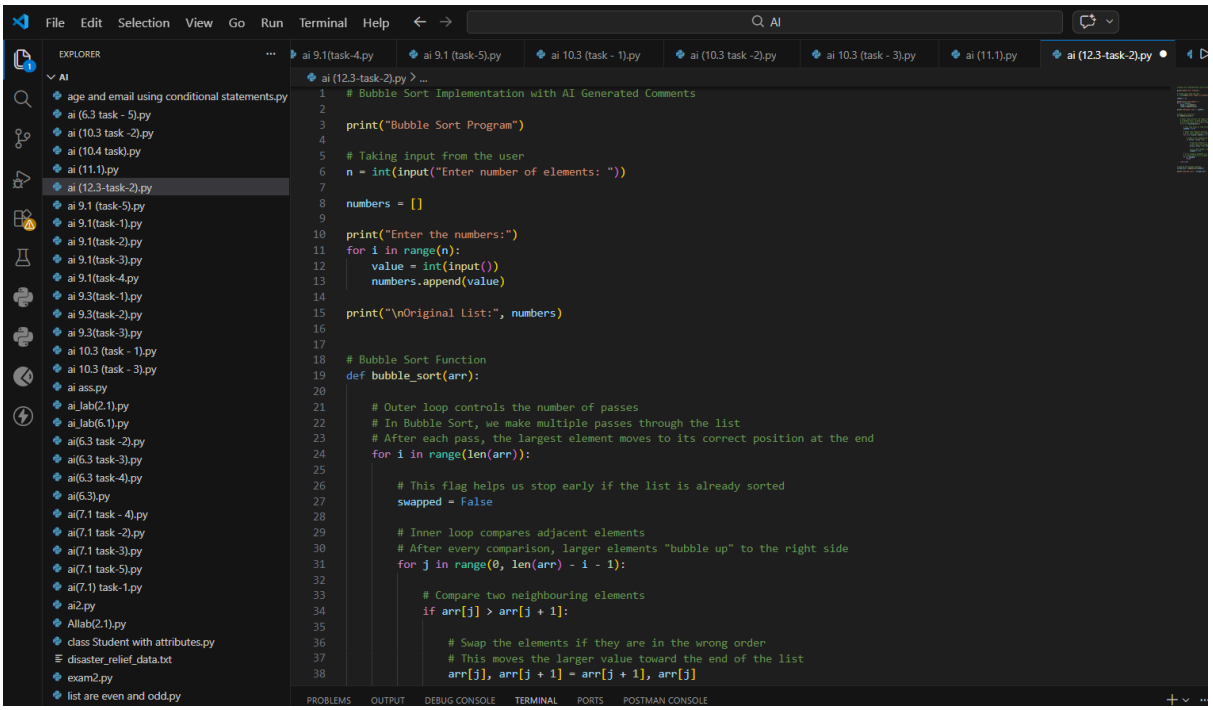
```
Run   Terminal   Help
...   PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

ents.py
PS C:\Users\Dell\Desktop\AI>  & 'c:\python314\pyth
ed\libs\debugpy\launcher' '52003' '--' 'c:\Users\D
SR University Placement Drive - Student Sorting Sy
Enter number of student records to generate for te

Generating student records...
Student records generated successfully.

Running Quick Sort...
Quick Sort completed in 1.4e-05 seconds

Running Quick Sort...
Quick Sort completed in 1.4e-05 seconds
Running Quick Sort...
Quick Sort completed in 1.4e-05 seconds
Quick Sort completed in 1.4e-05 seconds


Running Merge Sort...
Running Merge Sort...
Merge Sort completed in 1.4e-05 seconds
Merge Sort completed in 1.4e-05 seconds

Performance Comparison
Quick Sort Time : 1.4e-05 seconds
Merge Sort Time : 1.4e-05 seconds
Quick Sort Time : 1.4e-05 seconds
Merge Sort Time : 1.4e-05 seconds
Merge Sort Time : 1.4e-05 seconds

Top 10 Students Based on CGPA
1 - Student_1 | Roll: SRU1000 | CGPA: 8.68
2 - Student_2 | Roll: SRU1001 | CGPA: 5.2
PS C:\Users\Dell\Desktop\AI>
```

Task Description #2 (Implementing Bubble Sort with AI Comments)

Task: Write a Python implementation of Bubble Sort.

- **Instructions:**

- **Students implement Bubble Sort normally.**
- **Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).**
- **Request AI to provide time complexity analysis.**
- **Expected Output:**
- **A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis**



ai 9.1(task-4.py | ai 9.1 (task-5).py | ai 10.3 (task - 1).py | ai (10.3 task -2).py | ai 10.3 (task - 3).py | ai (11.1).py | ai (12.3-task-2).py

ai (12.3-task-2).py > ...

```python
def bubble_sort(arr):
        # This flag helps us stop early if the list is already sorted
        swapped = False

        # Inner loop compares adjacent elements
        # After every comparison, larger elements "bubble up" to the right side
        for j in range(0, len(arr) - i - 1):

            # Compare two neighbouring elements
            if arr[j] > arr[j + 1]:

                # Swap the elements if they are in the wrong order
                # This moves the larger value toward the end of the list
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

                # If a swap happens, we mark swapped as True
                swapped = True

        # If no swapping happened in a full pass,
        # it means the list is already sorted and we can stop early
        if not swapped:
            break

    return arr
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
Bubble Sort Program
n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '65141' '--' 'c:\Users\Dell\Desktop\AI\ai (12.3-task-2).py'
Bubble Sort Program
Enter number of elements: 3
Enter the numbers:
Enter the numbers:
1
1
2
2
3

Original List: [1, 2, 3]

Sorted List: [1, 2, 3]
PS C:\Users\Dell\Desktop\AI>
```

Task Description #3 Quick Sort and Merge Sort Comparison

- **Task: Implement Quick Sort and Merge Sort using recursion.**
- **Instructions:**
- **Provide AI with partially completed functions for recursion.**
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted lists.
- Expected Output:
- Working Quick Sort and Merge Sort implementations.
- AI-generated explanation of average, best, and worst-case complexities

ai 9.1 (task-5).py | ai 10.3 (task - 1).py | ai (10.3 task -2).py | ai 10.3 (task - 3).py | ai (11.1).py | ai (12.3-task-2).py | ai (12.3-task-

ai (12.3-task-3).py > ...

```python
# Quick Sort and Merge Sort Comparison using Recursion

import random
import time

print("Quick Sort and Merge Sort Comparison Program")

n = int(input("Enter number of elements to test sorting algorithms: "))


# Generate datasets
random_list = [random.randint(1, 1000) for _ in range(n)]
sorted_list = sorted(random_list)
reverse_list = sorted_list[::-1]


def quick_sort(arr):
    """
    Quick Sort Algorithm (Recursive)

    This function sorts a list using the Quick Sort method.
    It selects a pivot element and divides the list into three parts:
    elements smaller than pivot, equal to pivot, and greater than pivot.

    The function recursively sorts the smaller and larger lists
    until the entire list becomes sorted.
    """

    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```python

def merge_sort(arr):
    """
    Merge Sort Algorithm (Recursive)

    This function divides the list into two halves repeatedly
    until each part has one element. Then it merges the parts
    back together in sorted order.

    Merge Sort uses the divide and conquer technique.
    """

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)


def merge(left, right):
    """
    Helper function used by Merge Sort to merge two sorted lists
    into one sorted list.
    """

    merged = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] < right[j]:
            merged.append(left[i])
            i += 1
```

```python
def merge(left, right):
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged


def measure_time(sort_function, data):
    start = time.time()
    sort_function(data.copy())
    end = time.time()
    return end - start


print("\nTesting sorting performance on different datasets...\n")


datasets = {
    "Random List": random_list,
    "Sorted List": sorted_list,
    "Reverse Sorted List": reverse_list
}


for name, data in datasets.items():

    quick_time = measure_time(quick_sort, data)
    merge_time = measure_time(merge_sort, data)

    print(name)
    print("Quick Sort Time:", round(quick_time, 6), "seconds")
    print("Merge Sort Time:", round(merge_time, 6), "seconds")
    print()
```

```
PS C:\Users\Dell\Desktop\AI>  c:; cd 'c:\Users\Dell\Desktop\AI'; & 'c:\python314\python.exe' 'c:\Users\Dell\.vscode\extensions\ms-pytho
n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59324' '--' 'c:\Users\Dell\Desktop\AI\ai (12.3-task-3).py'
Testing sorting performance on different datasets...

Random List

Testing sorting performance on different datasets...

Random List
Testing sorting performance on different datasets...

Random List

Random List
Random List
Quick Sort Time: 1.8e-05 seconds
Quick Sort Time: 1.8e-05 seconds
Merge Sort Time: 1.8e-05 seconds
Merge Sort Time: 1.8e-05 seconds

Sorted List
Quick Sort Time: 3.4e-05 seconds
Merge Sort Time: 1.3e-05 seconds
Merge Sort Time: 1.3e-05 seconds

Reverse Sorted List
Quick Sort Time: 1.6e-05 seconds
Merge Sort Time: 3.3e-05 seconds

PS C:\Users\Dell\Desktop\AI>
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

- **Use AI to suggest the most efficient search and sort algorithms for this use case.**
- **Implement the recommended algorithms in Python.**
- **Justify the choice based on dataset size, update frequency, and performance requirements.**

Expected Output:

- **A table mapping operation → recommended algorithm → justification.**
- **Working Python functions for searching and sorting the inventory.**

ai 10.3 (task - 1).py | ai (10.3 task -2).py | ai 10.3 (task - 3).py | ai (11.1).py | ai (12.3-task-2).py | ai (12.3-task-3).py | ai(12.3

ai(12.3-task-4).py > ...

```python
# Inventory Management System - Searching and Sorting Products

import random

print("Retail Store Inventory Management System")

# Creating inventory list
products = []

n = int(input("Enter number of products to add: "))

# Taking product details from user
for i in range(n):

    print("\nEnter details for Product", i+1)

    product_id = int(input("Enter Product ID: "))
    name = input("Enter Product Name: ")
    price = float(input("Enter Product Price: "))
    quantity = int(input("Enter Stock Quantity: "))

    product = {
        "id": product_id,
        "name": name,
        "price": price,
        "quantity": quantity
    }

    products.append(product)


# Linear Search for Product Name
def search_by_name(product_list, target_name):
    """
    This function searches a product using its name.
    Linear Search is used because product names may not be sorted.
    """

    for product in product_list:
        if product["name"].lower() == target_name.lower():
            return product
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

ai 10.3 (task - 1).py | ai (10.3 task -2).py | ai 10.3 (task - 3).py | ai (11.1).py | ai (12.3-task-2).py | ai (12.3-ta

ai(12.3-task-4).py > ...

```python
    return None


# Binary Search for Product ID
def binary_search_by_id(sorted_products, target_id):
    """
    Binary Search works faster for large datasets.
    It requires the product list to be sorted by ID.
    """

    low = 0
    high = len(sorted_products) - 1

    while low <= high:

        mid = (low + high) // 2

        if sorted_products[mid]["id"] == target_id:
            return sorted_products[mid]

        elif sorted_products[mid]["id"] < target_id:
            low = mid + 1

        else:
            high = mid - 1

    return None


# Merge Sort for sorting by Price
def merge_sort_price(arr):
    """
    Merge Sort is used to sort products by price.
    It follows divide and conquer technique.
    """

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

ai(12.3-task-4).py > ...

```python
products_sorted_by_id = sorted(products, key=lambda x: x["id"])


# Searching product by ID
search_id = int(input("\nEnter Product ID to search: "))

result = binary_search_by_id(products_sorted_by_id, search_id)

if result:
    print("Product Found:", result)
else:
    print("Product not found")


# Searching product by Name
search_name = input("\nEnter Product Name to search: ")

result = search_by_name(products, search_name)

if result:
    print("Product Found:", result)
else:
    print("Product not found")


# Sorting by Price
sorted_by_price = merge_sort_price(products)

print("\nProducts Sorted by Price")

for p in sorted_by_price:
    print(p)


# Sorting by Quantity
sorted_by_quantity = quick_sort_quantity(products)

print("\nProducts Sorted by Quantity")

for p in sorted_by_quantity:
    print(p)
```

```
PS C:\Users\Dell\Desktop\AI>  c:; cd 'c:\Users\Dell\Desktop\AI'; & 'c:\python314\python.exe' 'c:\Users\Dell\.vscode\extensions\ms-pytho
n.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52081' '--' 'c:\Users\Dell\Desktop\AI\ai(12.3-task-4).py'
Retail Store Inventory Management System
Enter number of products to add: 2

Enter details for Product 1
Enter Product ID: 23456
Enter Product Name: pen
Enter Product Price: 200
Enter Stock Quantity: 50

Enter details for Product 2
Enter Product ID: 6789
Enter Product Name: book
Enter Product Price: 150
Enter Stock Quantity: 60

Enter Product ID to search: 6789
Product Found: {'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}

Enter Product Name to search: pen
Product Found: {'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}

Products Sorted by Price
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
Product Found: {'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}

Products Sorted by Price
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
Products Sorted by Price
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
{'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}
{'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}

Products Sorted by Quantity
Products Sorted by Quantity
{'id': 23456, 'name': 'pen', 'price': 200.0, 'quantity': 50}
{'id': 6789, 'name': 'book', 'price': 150.0, 'quantity': 60}
PS C:\Users\Dell\Desktop\AI>
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict

lookups) and analyze trade-offs

.

Go Run Terminal Help AI

ai (10.3 task -2).py | ai 10.3 (task - 3).py | ai (11.1).py | ai (12.3-task-2).py | ai (12.3-task-3).py | ai(12.3-task-4).py | ai(12.3-task-5).py | ai (10.4 task).py | ai ass.py

ai(12.3-task-5).py > ...

```python
# Real-Time Stock Data Sorting and Searching System

import random
import time

print("AI FinTech Lab - Stock Data Analysis Tool")

stocks = []

n = int(input("Enter number of stocks to simulate: "))

# Simulating stock data
for i in range(n):

    symbol = "STK" + str(i+1)

    opening_price = round(random.uniform(100, 500), 2)
    closing_price = round(random.uniform(100, 500), 2)

    percent_change = ((closing_price - opening_price) / opening_price) * 100

    stock = {
        "symbol": symbol,
        "open": opening_price,
        "close": closing_price,
        "change": percent_change
    }

    stocks.append(stock)


# Creating Hash Map for fast searching
stock_map = {}

for stock in stocks:
    stock_map[stock["symbol"]] = stock


# Heapify function used in Heap Sort
def heapify(arr, n, i):

    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left]["change"] > arr[largest]["change"]:
        largest = left

    if right < n and arr[right]["change"] > arr[largest]["change"]:
        largest = right
```

ai (10.3 task -2).py | ai 10.3 (task - 3).py | ai (11.1).py | ai (12.3-task-2).py | ai (12.3-task-3).py | ai(12.3-task-4).py | ai(12.3-task-5).py

ai(12.3-task-5).py > ...

```python
def heapify(arr, n, i):
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)


# Heap Sort for ranking stocks by percentage change
def heap_sort(arr):

    n = len(arr)

    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

    return arr


# Searching stock using Hash Map
def search_stock(symbol):
    """
    Dictionary lookup allows very fast search.
    Average time complexity is O(1).
    """

    return stock_map.get(symbol, None)


# Measuring Heap Sort performance
start = time.time()
sorted_stocks = heap_sort(stocks.copy())
heap_time = time.time() - start

print("\nStocks sorted by percentage change using Heap Sort")

for s in sorted_stocks:
    print(s)


# Searching for a stock symbol
symbol = input("\nEnter stock symbol to search: ")

result = search_stock(symbol)

if result:
    print("Stock Found:", result)
else:
    print("Stock not found")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

ai(12.3-task-5).py > ...

```python
        print("Stock not found")


# Comparing with Python built-in sorting
start = time.time()
builtin_sorted = sorted(stocks, key=lambda x: x["change"])
builtin_time = time.time() - start

print("\nPerformance Comparison")
print("Heap Sort Time:", round(heap_time, 6), "seconds")
print("Python sorted() Time:", round(builtin_time, 6), "seconds")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Dell\Desktop\AI>  c:; cd 'c:\Users\Dell\Desktop\AI'; & 'c:\python314\python.exe' 'c:\Users\Dell\.vscode\extensions\ms-python.debugpy-2025.18.0-wi
 '--' 'c:\Users\Dell\Desktop\AI\ai(12.3-task-5).py'
AI FinTech Lab - Stock Data Analysis Tool
Enter number of stocks to simulate: 3

Stocks sorted by percentage change using Heap Sort
{'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
{'symbol': 'STK3', 'open': 496.02, 'close': 218.76, 'change': -55.896939639530665}
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
AI FinTech Lab - Stock Data Analysis Tool
Enter number of stocks to simulate: 3

Stocks sorted by percentage change using Heap Sort
{'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
{'symbol': 'STK3', 'open': 496.02, 'close': 218.76, 'change': -55.896939639530665}
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
Stocks sorted by percentage change using Heap Sort
{'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
{'symbol': 'STK3', 'open': 496.02, 'close': 218.76, 'change': -55.896939639530665}
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
{'symbol': 'STK2', 'open': 383.79, 'close': 426.61, 'change': 11.157143229370226}

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison

Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}
```

Navigate to Command
Scroll to Previous Command (Ctrl+
Scroll to Next Command (Ctrl+Do

```
Enter stock symbol to search: STK1
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
Stock Found: {'symbol': 'STK1', 'open': 492.0, 'close': 182.45, 'change': -62.916666666666664}

Performance Comparison
Heap Sort Time: 4.6e-05 seconds

Performance Comparison
Heap Sort Time: 4.6e-05 seconds
Heap Sort Time: 4.6e-05 seconds
Python sorted() Time: 3.7e-05 seconds
PS C:\Users\Dell\Desktop\AI>
```